SME-I Manager 数据接口与源代码

项目信息	内容
项目名称	中小型企业经济智慧管理工具（SME-I Manager）
后端技术栈	Node.js 24 + Express 4 + node:sqlite（Node 内置，零外部数据库依赖）
依赖数量	2 个（express、cors）
接口总数	**39 个**（38 个业务接口 + 1 个健康检查）
代码规模	23 个源文件，1755 行
版本 / 提交	v0.3 / `e6765ec`（共 4 个提交）
生成时间	2026-10-23 01:13

目录

1. 接口总表（39 个）
2. 如何运行
3. 源代码清单
4. 源代码正文

1. 接口总表（39 个）

基址 `http://<IP>:3000`　鉴权 `Authorization: Bearer <token>`　金额单位「元」　月份 `YYYY-MM`
⭐=MVP 核心

#	模块	方法	路径	用途
1	系统	GET	`/api/health`	健康检查（免鉴权）
2	认证	POST	`/api/auth/login`	登录，返回 token
3	认证	GET	`/api/auth/me`	当前用户与角色
4	认证	POST	`/api/auth/logout`	登出
5	记账	GET	`/api/transactions`	流水列表（分页+筛选）
6	记账	GET	⭐ `/api/transactions/summary`	汇总：收入/支出/毛利/待归类
7	记账	GET	`/api/transactions/pending`	待确认列表
8	记账	POST	⭐ `/api/transactions`	手工录入
9	记账	PUT	`/api/transactions/:id`	修改
10	记账	DELETE	`/api/transactions/:id`	删除

#	模块	方法	路径	用途
11	记账	POST	⭐ `/api/transactions/:id/confirm`	人工确认归类
12	采集	POST	⭐ `/api/import/bills`	上传账单 CSV（微信 UTF-8 / 支付宝 GBK）
13	采集	GET	`/api/import`	导入历史
14	采集	GET	`/api/import/:jobId`	单次导入结果
15	分类	GET	`/api/categories`	分类列表
16	分类	POST	`/api/categories`	新建分类
17	商品	GET	`/api/products`	商品列表（含毛利率）
18	商品	POST	`/api/products`	新建
19	商品	PUT	`/api/products/:id`	修改
20	商品	DELETE	`/api/products/:id`	删除
21	分析	GET	⭐ `/api/analytics/overview`	四张卡 + 环比 + 同比
22	分析	GET	⭐ `/api/analytics/trend`	营收趋势折线
23	分析	GET	⭐ `/api/analytics/cost-structure`	成本结构饼图
24	分析	GET	`/api/analytics/product-margin`	商品毛利
25	分析	GET	`/api/analytics/compare`	同环比对比表
26	问答	POST	⭐ `/api/ai/ask`	智能问答（归因+建议+计算依据）
27	预警	GET	`/api/alerts`	预警列表
28	预警	GET	`/api/alerts/settings`	读阈值
29	预警	PUT	`/api/alerts/settings`	改阈值
30	预警	POST	`/api/alerts/:id/ignore`	忽略预警
31	预警	GET	`/api/alerts/unread-count`	未读数
32	票据	POST	`/api/ocr/receipt`	票据识别（占位实现）
33	票据	POST	`/api/ocr/receipt/:id/confirm`	确认入账
34	票据	GET	`/api/ocr/receipts`	票据历史
35	系统	GET	`/api/system/members`	成员列表
36	系统	POST	`/api/system/members/invite`	邀请成员（老板）
37	系统	PUT	`/api/system/members/:id`	改角色（老板）
38	系统	POST	`/api/system/backup`	全库备份（老板）
39	系统	GET	`/api/system/export`	导出 CSV

模块分布：认证 3 | 记账 7 | 采集 3 | 分类 2 | 商品 4 | 分析 5 | 问答 1 | 预警 5 | 票据 3 | 系统 5 | 健康检查 1 = 39

---

2. 如何运行

```
unzip sme-i-api-v0.3.zip && cd sme-i-api
npm install && npm run seed && npm start
# 打开 http://127.0.0.1:3000/api/health
```

演示账号：**张总 / 13800000001 / 123456**（老板）、**李会计 / 13800000002 / 123456**（店员）

---

3. 源代码清单

#	文件	行数	职责
1	`package.json`	20	依赖与启动脚本
2	`.env.example`	15	环境变量模板
3	`.gitignore`	8	Git 忽略规则
4	`Dockerfile`	22	容器镜像构建
5	`start.sh`	15	脚本部署入口
6	`src/index.js`	54	服务入口：中间件、挂路由、监听
7	`src/db.js`	119	建表语句 + 数据库连接
8	`src/seed.js`	178	可复现的演示数据
9	`src/lib/money.js`	21	分/元转换、百分比、环比
10	`src/lib/metrics.js`	182	★ 指标计算（收入/成本/毛利/毛利率/环比/同比/分类环比）
11	`src/lib/csv.js`	180	★ 微信/支付宝账单解析（编码探测 + 跳过说明行）
12	`src/lib/classifier.js`	46	关键词自动分类
13	`src/lib/auth.js`	48	HMAC token + 鉴权 / 权限中间件
14	`src/routes/auth.js`	34	认证接口（3 个）
15	`src/routes/transactions.js`	169	★ 记账接口（7 个）
16	`src/routes/import.js`	105	★ 账单导入接口（3 个）
17	`src/routes/categories.js`	34	分类接口（2 个）
18	`src/routes/products.js`	58	商品接口（4 个）
19	`src/routes/analytics.js`	48	★ 经营分析接口（5 个）
20	`src/routes/ai.js`	167	★ 智能问答接口（先算数再解释）
21	`src/routes/alerts.js`	52	预警接口（5 个）
22	`src/routes/ocr.js`	85	票据识别接口（3 个）
23	`src/routes/system.js`	95	系统管理接口（5 个）
	合计	**1755**	

4. 源代码正文

4.1 `package.json`（20 行）

```
{
  "name": "sme-i-api",
  "version": "0.1.0",
  "private": true,
  "description": "SME-I Manager 后端数据接口（小微企业经营管理工具）",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "node --watch src/index.js",
    "seed": "node src/seed.js",
    "reset": "rm -rf data && node src/seed.js"
  },
  "engines": {
    "node": ">=22.5.0"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^4.21.2"
  }
}
```

4.2 `.env.example`（15 行）

```
# ===== 服务 =====
PORT=3000
# 数据库文件路径（SQLite）。部署到容器时请挂载持久化卷，或改用 RDS
DB_PATH=./data/sme.db

# ===== 认证 =====
# 生成方式: node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
AUTH_SECRET=dev-only-secret-please-change-in-production

# ===== 大模型（可选）=====
# 不填 API Key 时，/api/ai/ask 会自动降级为「内置规则引擎」，演示照样能用
LLM_BASE_URL=https://api.deepseek.com/v1
LLM_API_KEY=
LLM_MODEL=deepseek-chat
LLM_TIMEOUT_MS=20000
```

4.3 `.gitignore`（8 行）

```
node_modules/
data/
.env
*.log
.DS_Store
~$*
.~*
npm-debug.log*
```

4.4 `Dockerfile`（22 行）

```
# SME-I Manager 后端数据接口
# Node 24 内置 node:sqlite，无需安装数据库驱动
FROM node:24-alpine

WORKDIR /app

# 先装依赖，利用 Docker 层缓存
COPY package*.json ./
RUN npm ci --omit=dev || npm install --omit=dev

COPY . .

ENV PORT=3000
ENV DB_PATH=/app/data/sme.db
EXPOSE 3000

# 健康检查：CodeArts 部署服务 / K8s 靠它判断实例是否存活
HEALTHCHECK --interval=30s --timeout=5s --start-period=15s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/api/health || exit 1

# 首次启动若无数据库则自动建库并灌入演示数据
CMD ["sh", "-c", "node src/seed.js && node src/index.js"]
```

4.5 `start.sh`（15 行）

```
#!/bin/sh
# 用于 CodeArts Deploy 的自定义脚本部署（不走容器时）
set -e

echo "==> 检查 Node 版本"
node -v

echo "==> 安装依赖"
npm ci --omit=dev || npm install --omit=dev

echo "==> 初始化数据库（已存在则跳过，不会覆盖数据）"
node src/seed.js

echo "==> 启动服务（端口 ${PORT:-3000}）"
exec node src/index.js
```

4.6 `src/index.js`（54 行）

```
'use strict';
/**
 * 服务入口。
 * 启动顺序：建表 -> 注册中间件 -> 挂路由 -> 监听
 *
 * 端口从 process.env.PORT 读（云上必须这样，不能硬编码），
 * 日志打到 stdout（云平台靠它收日志）。
 */
const express = require('express');
const cors = require('cors');
const { initSchema } = require('./db');

const PORT = Number(process.env.PORT || 3000);
const app = express();
```

```
initSchema();

app.use(cors());
app.use(express.json({ limit: '2mb' }));
// 账单 CSV 以原始字节上传（前端用 text/plain 或 application/octet-stream 发送文件内容）
app.use('/api/import/bills', express.raw({ type: '*/*', limit: '20mb' }));
// 票据图片同理。⚠️ 只匹配二进制类型，避免把 /receipt/:id/confirm 的 JSON 也当 Buffer 吃掉
app.use('/api/ocr/receipt', express.raw({ type: ['application/octet-stream', 'image/*',
'application/pdf'], limit: '20mb' }));

// ---- 健康检查：CodeArts 部署 / K8s / 负载均衡都靠它 ----
app.get('/api/health', (req, res) => {
  res.json({ ok: true, service: 'sme-i-api', version: '0.1.0', node: process.version, time: new
Date().toISOString() });
});

app.use('/api/auth', require('./routes/auth'));
app.use('/api/transactions', require('./routes/transactions'));
app.use('/api/import', require('./routes/import'));
app.use('/api/categories', require('./routes/categories'));
app.use('/api/products', require('./routes/products'));
app.use('/api/analytics', require('./routes/analytics'));
app.use('/api/ai', require('./routes/ai'));
app.use('/api/alerts', require('./routes/alerts'));
app.use('/api/ocr', require('./routes/ocr'));
app.use('/api/system', require('./routes/system'));

// 404
app.use((req, res) => res.status(404).json({ error: 'not_found', path: req.path }));

// 统一错误处理（必须放最后，4 个参数才是错误中间件）
app.use((err, req, res, next) => {
  console.error('[error]', req.method, req.path, err.message);
  res.status(err.status || 500).json({ error: err.code || 'internal_error', message: err.message });
});

app.listen(PORT, '0.0.0.0', () => {
  console.log(`[sme-i-api] listening on http://0.0.0.0:${PORT}`);
  console.log(`[sme-i-api] health : http://127.0.0.1:${PORT}/api/health`);
  console.log(`[sme-i-api] docs   : see API.md`);
});
```

4.7 `src/db.js`（119 行）

```
'use strict';
/**
 * 数据库层：使用 Node 24 内置的 node:sqlite，零外部依赖。
 * 金额统一用「整数分」存储，避免浮点误差。
 *
 * 换成华为云 RDS(MySQL) 时，只需替换本文件里的 SQL 方言和驱动，
 * 上层 routes / lib 都不用改（这就是分层的好处）。
 */
const path = require('node:path');
const fs = require('node:fs');
const { DatabaseSync } = require('node:sqlite');

const DB_PATH = process.env.DB_PATH || path.join(__dirname, '..', 'data', 'sme.db');

fs.mkdirSync(path.dirname(DB_PATH), { recursive: true });

const db = new DatabaseSync(DB_PATH);
db.exec('PRAGMA journal_mode = WAL;');
db.exec('PRAGMA foreign_keys = ON;');
```

```
const SCHEMA = `
CREATE TABLE IF NOT EXISTS users (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  name          TEXT    NOT NULL,
  phone         TEXT,
  industry      TEXT,
  role          TEXT    NOT NULL DEFAULT 'staff',   -- owner(老板) | staff(店员)
  password_hash TEXT    NOT NULL,
  created_at    TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);

CREATE TABLE IF NOT EXISTS categories (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id    INTEGER NOT NULL REFERENCES users(id),
  name       TEXT    NOT NULL,
  kind       TEXT    NOT NULL,                       -- income | expense
  color      TEXT,
  created_at TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);

CREATE TABLE IF NOT EXISTS transactions (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id      INTEGER NOT NULL REFERENCES users(id),
  date         TEXT    NOT NULL,                    -- YYYY-MM-DD
  type         TEXT    NOT NULL,                    -- income | expense
  amount_cents INTEGER NOT NULL,                    -- 金额（分），永远为正
  category_id  INTEGER REFERENCES categories(id),
  summary      TEXT,
  note         TEXT,
  source       TEXT    NOT NULL DEFAULT 'manual',    -- manual | wechat | alipay | ocr
  status       TEXT    NOT NULL DEFAULT 'confirmed', -- confirmed | pending
  created_at   TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);
CREATE INDEX IF NOT EXISTS idx_tx_user_date ON transactions(user_id, date);
CREATE INDEX IF NOT EXISTS idx_tx_user_status ON transactions(user_id, status);

CREATE TABLE IF NOT EXISTS products (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id     INTEGER NOT NULL REFERENCES users(id),
  name        TEXT    NOT NULL,
  price_cents INTEGER NOT NULL,                     -- 售价
  cost_cents  INTEGER NOT NULL,                     -- 成本价
  created_at  TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);

CREATE TABLE IF NOT EXISTS import_jobs (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id    INTEGER NOT NULL REFERENCES users(id),
  platform   TEXT    NOT NULL,                      -- wechat | alipay
  filename   TEXT,
  total      INTEGER NOT NULL DEFAULT 0,
  categorized INTEGER NOT NULL DEFAULT 0,
  pending    INTEGER NOT NULL DEFAULT 0,
  status     TEXT    NOT NULL DEFAULT 'done',       -- running | done | failed
  message    TEXT,
  created_at TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);

CREATE TABLE IF NOT EXISTS alerts (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id    INTEGER NOT NULL REFERENCES users(id),
  type       TEXT    NOT NULL,                      -- cost_ratio | revenue_mom | no_flow
  level      TEXT    NOT NULL DEFAULT 'info',       -- high | medium | info
  title      TEXT    NOT NULL,
```

```
  detail     TEXT,
  threshold  TEXT,
  status     TEXT    NOT NULL DEFAULT 'open',        -- open | ignored
  pushed_at  TEXT,
  created_at TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);

CREATE TABLE IF NOT EXISTS alert_settings (
  user_id            INTEGER PRIMARY KEY REFERENCES users(id),
  cost_ratio         REAL NOT NULL DEFAULT 40,
  revenue_volatility REAL NOT NULL DEFAULT 5,
  margin_floor       REAL NOT NULL DEFAULT 30
);

CREATE TABLE IF NOT EXISTS receipts (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id         INTEGER NOT NULL REFERENCES users(id),
  image_path      TEXT,
  merchant        TEXT,
  date            TEXT,
  amount_cents    INTEGER,
  category        TEXT,
  note            TEXT,
  confidence_json TEXT,
  status          TEXT NOT NULL DEFAULT 'pending',   -- pending | confirmed
  created_at      TEXT NOT NULL DEFAULT (datetime('now','localtime'))
);
`;

function initSchema() {
  db.exec(SCHEMA);
  return db;
}

module.exports = { db, initSchema, DB_PATH };
```

4.8 `src/seed.js`（178 行）

```
'use strict';
/**
 * 初始化数据库 + 灌入演示数据。
 *
 * 演示数字已与效果图对齐，可用来验证指标口径：
 *   2026-06  营业额 128,400  成本 82,150  利润 46,250  毛利率 36.0%
 *   2026-05  营业额 135,200  成本 82,337  利润 52,863  毛利率 39.1%
 *   → 环比: 营业额 -5.0%  原材料 +12.2%  毛利率 -3.1 个点
 *   成本结构: 原材料 45% / 人工 22% / 房租 18% / 其他 15%
 *
 * 用法:
 *   node src/seed.js               # 建表；库为空时灌入演示数据
 *   node src/seed.js --init-only   # 只建表，不灌数据
 *   node src/seed.js --force       # 清空后重新灌入
 */
const { db, initSchema } = require('./db');
const { hashPassword } = require('./lib/auth');

const toCents = (v) => Math.round(Number(v) * 100);

// ---------- 可复现的伪随机（保证每次生成的数据一样） ----------
let _seed = 20260615;
const rnd = () => (_seed = (_seed * 1103515245 + 12345) % 2147483648) / 2147483648;

/** 把总金额按权重拆成 n 份（整数分，最后一份补齐差额，保证总和精确） */
```

```
function split(totalCents, weights) {
  const sum = weights.reduce((a, b) => a + b, 0);
  const out = weights.map((w) => Math.floor((totalCents * w) / sum));
  out[out.length - 1] += totalCents - out.reduce((a, b) => a + b, 0);
  return out;
}

const CATEGORIES = [
  ['营业收入', 'income'],
  ['退款', 'expense'],
  ['原材料', 'expense'],
  ['人工', 'expense'],
  ['房租', 'expense'],
  ['其他', 'expense'],
];

/**
 * 每个月的数据：[月份, 收入(元), 原材料, 人工, 房租, 其他]
 * 注意 2026-05 / 2026-06 的原材料金额是专门为效果图对齐的。
 */
const MONTHS = [
  ['2025-06', 130800, 31000, 18000, 14000, 16900],
  ['2026-01', 112000, 28000, 17000, 14000, 15000],
  ['2026-02',  98000, 26000, 16000, 14000, 12000],
  ['2026-03', 121000, 30000, 17500, 14000, 17500],
  ['2026-04', 127000, 31500, 18000, 14000, 17500],
  ['2026-05', 135200, 32900, 18073, 14787, 16577],
  ['2026-06', 128400, 36900, 18073, 14787, 12390],
];

const EXPENSE_LABELS = {
  原材料: ['采购原材料', '采购咖啡豆', '进货-奶基底', '进货-杯材', '进货-水果'],
  人工: ['员工薪资', '兼职时薪', '社保公积金'],
  房租: ['门店房租', '物业费', '水电费'],
  其他: ['设备维修', '外卖平台服务费', '门店物料', '团建'],
};

function main() {
  const args = process.argv.slice(2);
  initSchema();

  const hasData = Number(db.prepare('SELECT COUNT(*) c FROM users').get().c) > 0;

  if (args.includes('--force')) {
    console.log('[seed] --force：清空原有数据');
    for (const t of ['receipts', 'alerts', 'alert_settings', 'import_jobs', 'transactions',
'products', 'categories', 'users']) {
      db.exec(`DELETE FROM ${t}`);
    }
  } else if (args.includes('--init-only')) {
    console.log('[seed] 仅建表，不灌数据');
    return;
  } else if (hasData) {
    console.log('[seed] 数据库已有数据，跳过。要重建请执行：npm run reset');
    return;
  }

  db.exec('BEGIN');
  try {
    // ---------- 用户 ----------
    const insUser = db.prepare('INSERT INTO users(name,phone,industry,role,password_hash)
VALUES(?,?,?,?,?)');
    const ownerId = Number(insUser.run('张总', '13800000001', '餐饮', 'owner',
hashPassword('123456')).lastInsertRowid);
    insUser.run('李会计', '13800000002', '餐饮', 'staff', hashPassword('123456'));
```

```
    // ---------- 分类 ----------
    const insCat = db.prepare('INSERT INTO categories(user_id,name,kind) VALUES(?,?,?)');
    const catId = {};
    for (const [name, kind] of CATEGORIES) catId[name] = Number(insCat.run(ownerId, name,
kind).lastInsertRowid);

    // ---------- 交易 ----------
    const insTx = db.prepare(
      `INSERT INTO
transactions(user_id,date,type,amount_cents,category_id,summary,note,source,status)
       VALUES(?,?,?,?,?,?,?,?,?)`
    );
    const pad = (n) => String(n).padStart(2, '0');

    for (const [month, income, mat, labor, rent, other] of MONTHS) {
      // 收入：老月份用 4 条汇总，2026-06 用 25 条日流水（让列表看起来真实）
      if (month === '2026-06') {
        const n = 25;
        const w = Array.from({ length: n }, () => 0.6 + rnd() * 0.8);
        const parts = split(toCents(income), w);
        for (let i = 0; i < n; i++) {
          const day = 1 + Math.floor((i * 30) / n);
          insTx.run(ownerId, `${month}-${pad(Math.min(day, 30))}`, 'income', parts[i],
            catId['营业收入'], '门店营业额', null, i % 5 === 0 ? 'wechat' : (i % 7 === 0 ? 'alipay' :
'manual'), 'confirmed');
        }
      } else {
        const parts = split(toCents(income), [1, 1, 1, 1]);
        for (let i = 0; i < 4; i++) {
          insTx.run(ownerId, `${month}-${pad(5 + i * 7)}`, 'income', parts[i],
            catId['营业收入'], '门店营业额（周汇总）', null, i % 2 ? 'wechat' : 'alipay', 'confirmed');
        }
      }

      // 支出：每个科目 2~5 条，按权重拆分
      const expensePlan = [['原材料', mat], ['人工', labor], ['房租', rent], ['其他', other]];
      for (const [cat, amount] of expensePlan) {
        if (!amount) continue;
        const labels = EXPENSE_LABELS[cat];
        const n = Math.min(labels.length, cat === '房租' ? 3 : 4);
        const w = Array.from({ length: n }, () => 0.7 + rnd() * 0.6);
        const parts = split(toCents(amount), w);
        for (let i = 0; i < n; i++) {
          const day = 3 + Math.floor((i * 24) / n);
          insTx.run(ownerId, `${month}-${pad(Math.min(day, 28))}`, 'expense', parts[i],
            catId[cat], labels[i % labels.length], null,
            cat === '原材料' ? (i % 2 ? 'alipay' : 'manual') : 'manual', 'confirmed');
        }
      }
    }

    // ---------- 待确认账单 34 笔（对应「待归类 34 笔」） ----------
    const pendingLabels = ['未知收款方', '扫码支付', '线上订单', '供应商A', '供应商B'];
    for (let i = 0; i < 34; i++) {
      const cents = toCents(8 + Math.floor(rnd() * 300));
      insTx.run(ownerId, `2026-06-${pad(1 + (i % 28))}`, i % 3 === 0 ? 'income' : 'expense', cents,
        null, pendingLabels[i % pendingLabels.length], '自动分类未命中，待人工确认',
        i % 2 ? 'wechat' : 'alipay', 'pending');
    }

    // ---------- 商品（毛利率对齐效果图） ----------
    const insProd = db.prepare('INSERT INTO products(user_id,name,price_cents,cost_cents)
VALUES(?,?,?,?)');
    insProd.run(ownerId, '招牌奶茶', toCents(18.0), toCents(6.84));   // 62%
```

```
    insProd.run(ownerId, '双人套餐', toCents(68.0), toCents(35.36));  // 48%
    insProd.run(ownerId, '咖啡', toCents(16.0), toCents(4.64));       // 71%

    // ---------- 预警 ----------
    const insAlert = db.prepare(
      'INSERT INTO alerts(user_id,type,level,title,detail,threshold,pushed_at) VALUES(?,?,?,?,?,?,?)'
    );
    insAlert.run(ownerId, 'cost_ratio', 'high', '原材料成本占比 45%',
      '已连续 2 周超标', '阈值 40%', '今天 09:20');
    insAlert.run(ownerId, 'revenue_mom', 'medium', '营业额环比下降 5%',
      '低于近 3 个月均值', '阈值 5%', '今天 09:20');
    insAlert.run(ownerId, 'no_flow', 'info', '本周有 1 天无流水',
      '可能漏记，建议补录', null, '昨天 18:05');

    // ---------- 预警阈值 ----------
    db.prepare('INSERT INTO alert_settings(user_id,cost_ratio,revenue_volatility,margin_floor)
VALUES(?,?,?,?)')
      .run(ownerId, 40, 5, 30);

    db.exec('COMMIT');
  } catch (e) {
    db.exec('ROLLBACK');
    throw e;
  }

  const txCount = Number(db.prepare('SELECT COUNT(*) c FROM transactions').get().c);
  console.log(`[seed] 完成：用户 2 个，交易 ${txCount} 条，商品 3 个，预警 3 条`);
  console.log('[seed] 演示账号：张总 / 13800000001 / 123456（老板）；李会计 / 13800000002 / 123456（店
员）');
}

main();
```

4.9 `src/lib/money.js`（21 行）

```
'use strict';
/**
 * 金额工具：数据库里永远是「整数分」，只在输出给前端时才转「元」。
 * 这是新手第一大坑 —— 用浮点数存钱，算着算着就多了 0.01。
 */
const toCents = (v) => {
  const n = Number(v);
  if (!Number.isFinite(n)) throw new Error('金额不合法: ' + v);
  return Math.round(n * 100);
};

const toYuan = (cents) => Number((Number(cents || 0) / 100).toFixed(2));

const pct = (num, den, digits = 1) =>
  den ? Number(((num / den) * 100).toFixed(digits)) : 0;

/** 环比/同比：分母为 0 时返回 null，前端显示 "—" */
const growth = (cur, prev, digits = 1) =>
  prev ? Number((((cur - prev) / prev) * 100).toFixed(digits)) : null;

module.exports = { toCents, toYuan, pct, growth };
```

4.10 `src/lib/metrics.js`（182 行）

```
'use strict';
/**
```

```
 * 指标计算层 —— 本项目的核心，对应 PRD「系统算数、模型解释」里的「系统算数」。
 * 所有数字都在这里算出来，大模型只负责解释，绝不让它算数。
 *
 * 口径（已与效果图数字核对）：
 *   收入   = SUM(income, 已确认)
 *   成本   = SUM(expense, 已确认)
 *   毛利   = 收入 − 成本
 *   毛利率 = 毛利 / 收入
 *   环比   = (本月 − 上月) / 上月
 *   同比   = (本月 − 去年同月) / 去年同月
 *   待归类 = status='pending' 的笔数（不计入指标）
 */
const { toYuan, pct, growth } = require('./money');

/** '2026-06' -> ['2026-06-01','2026-07-01') */
function monthRange(month) {
  const [y, m] = String(month).split('-').map(Number);
  const ny = m === 12 ? y + 1 : y;
  const nm = m === 12 ? 1 : m + 1;
  return [`${y}-${String(m).padStart(2, '0')}-01`, `${ny}-${String(nm).padStart(2, '0')}-01`];
}

/** 月份偏移：shiftMonth('2026-01', -1) -> '2025-12' */
function shiftMonth(month, delta) {
  const [y, m] = String(month).split('-').map(Number);
  const d = new Date(Date.UTC(y, m - 1 + delta, 1));
  return `${d.getUTCFullYear()}-${String(d.getUTCMonth() + 1).padStart(2, '0')}`;
}

function sumType(db, userId, from, to, type) {
  const row = db
    .prepare(
      `SELECT COALESCE(SUM(amount_cents),0) AS s FROM transactions
        WHERE user_id=? AND status='confirmed' AND type=? AND date>=? AND date<?`
    )
    .get(userId, type, from, to);
  return Number(row.s || 0);
}

function pendingCount(db, userId, month) {
  const [from, to] = monthRange(month);
  const row = db
    .prepare(
      `SELECT COUNT(*) AS c FROM transactions
        WHERE user_id=? AND status='pending' AND date>=? AND date<?`
    )
    .get(userId, from, to);
  return Number(row.c || 0);
}

/** 某个月的原始指标（分） */
function monthMetrics(db, userId, month) {
  const [from, to] = monthRange(month);
  const revenue = sumType(db, userId, from, to, 'income');
  const cost = sumType(db, userId, from, to, 'expense');
  return { month, revenue, cost, profit: revenue - cost };
}

/** GET /api/analytics/overview */
function overview(db, userId, month) {
  const cur = monthMetrics(db, userId, month);
  const prev = monthMetrics(db, userId, shiftMonth(month, -1));
  const lastYear = monthMetrics(db, userId, shiftMonth(month, -12));

  const wrap = (m) => ({
```

```
    revenue: toYuan(m.revenue),
    cost: toYuan(m.cost),
    profit: toYuan(m.profit),
    margin: pct(m.profit, m.revenue),
  });

  const c = wrap(cur), p = wrap(prev), ly = wrap(lastYear);

  return {
    month,
    ...c,
    mom: {
      revenue: growth(c.revenue, p.revenue),
      cost: growth(c.cost, p.cost),
      profit: growth(c.profit, p.profit),
      margin: Number((c.margin - p.margin).toFixed(1)), // 百分点差，不是百分比
    },
    yoy: {
      revenue: growth(c.revenue, ly.revenue),
      cost: growth(c.cost, ly.cost),
      profit: growth(c.profit, ly.profit),
      margin: Number((c.margin - ly.margin).toFixed(1)),
    },
    prev: { month: prev.month, ...p },
    pendingCount: pendingCount(db, userId, month),
  };
}

/** GET /api/analytics/trend?from=2026-01&to=2026-06 */
function trend(db, userId, fromMonth, toMonth) {
  const out = [];
  let m = fromMonth;
  for (let i = 0; i < 36; i++) {
    const r = monthMetrics(db, userId, m);
    out.push({
      month: m,
      revenue: toYuan(r.revenue),
      cost: toYuan(r.cost),
      profit: toYuan(r.profit),
    });
    if (m === toMonth) break;
    m = shiftMonth(m, 1);
  }
  return out;
}

/** GET /api/analytics/cost-structure */
function costStructure(db, userId, month) {
  const [from, to] = monthRange(month);
  const rows = db
    .prepare(
      `SELECT COALESCE(c.name,'其他') AS name, SUM(t.amount_cents) AS s
         FROM transactions t LEFT JOIN categories c ON c.id = t.category_id
        WHERE t.user_id=? AND t.status='confirmed' AND t.type='expense'
          AND t.date>=? AND t.date<?
        GROUP BY name ORDER BY s DESC`
    )
    .all(userId, from, to);

  const total = rows.reduce((a, r) => a + Number(r.s), 0);
  return {
    month,
    total: toYuan(total),
    items: rows.map((r) => ({
      name: r.name,
```

```
      amount: toYuan(r.s),
      ratio: pct(Number(r.s), total),
    })),
  };
}

/** GET /api/analytics/product-margin */
function productMargin(db, userId) {
  const rows = db.prepare(`SELECT * FROM products WHERE user_id=? ORDER BY id`).all(userId);
  return rows.map((p) => ({
    id: p.id,
    name: p.name,
    price: toYuan(p.price_cents),
    cost: toYuan(p.cost_cents),
    margin: pct(p.price_cents - p.cost_cents, p.price_cents),
  }));
}

/** 分类级环比 —— 用于「原材料成本上涨 12%」这类归因（不是总成本） */
function categorySeries(db, userId, month, prevMonth) {
  const one = (m) => {
    const [from, to] = monthRange(m);
    const rows = db
      .prepare(
        `SELECT COALESCE(c.name,'其他') AS name, SUM(t.amount_cents) AS s
           FROM transactions t LEFT JOIN categories c ON c.id = t.category_id
          WHERE t.user_id=? AND t.status='confirmed' AND t.type='expense'
            AND t.date>=? AND t.date<?
          GROUP BY name`
      )
      .all(userId, from, to);
    return Object.fromEntries(rows.map((r) => [r.name, Number(r.s)]));
  };
  const cur = one(month);
  const prev = one(prevMonth);
  const names = [...new Set([...Object.keys(cur), ...Object.keys(prev)])];
  return names
    .map((n) => ({
      name: n,
      current: toYuan(cur[n] || 0),
      prev: toYuan(prev[n] || 0),
      mom: growth(cur[n] || 0, prev[n] || 0),
    }))
    .sort((a, b) => b.current - a.current);
}

module.exports = {
  monthRange, shiftMonth, monthMetrics, overview, trend, costStructure, productMargin,
  pendingCount, categorySeries,
};
```

4.11 `src/lib/csv.js`（180 行）

```
'use strict';
/**
 * 微信支付 / 支付宝 账单 CSV 解析器。
 *
 * ⚠️ 这是整个项目最容易出错的地方，两个平台的格式完全不同：
 *
 *   微信支付：
 *     - 编码 UTF-8（带 BOM）
 *     - 前面约 16 行是说明文字，"交易时间"那行才是表头
 *     - 金额列名 "金额(元)"
```

```
 *
 *   支付宝：
 *     - 编码 GBK（必须转码，否则中文全是乱码）
 *     - 前面约 24 行说明文字
 *     - 金额列名 "金额"
 *
 * 用法：parseBill(buffer, 'wechat' | 'alipay')
 */
const { toCents } = require('./money');

/** 自动探测编码：先严格试 UTF-8，失败则按 GBK 解 */
function decodeBuffer(buf) {
  if (!Buffer.isBuffer(buf)) buf = Buffer.from(String(buf), 'utf8');
  // 去 BOM
  if (buf.length >= 3 && buf[0] === 0xef && buf[1] === 0xbb && buf[2] === 0xbf) {
    buf = buf.subarray(3);
  }
  try {
    return new TextDecoder('utf-8', { fatal: true }).decode(buf);
  } catch {
    try {
      return new TextDecoder('gbk').decode(buf);
    } catch {
      return new TextDecoder('utf-8').decode(buf);
    }
  }
}

function splitLines(text) {
  return String(text).replace(/\r\n?/g, '\n').split('\n');
}

/** 支持双引号包裹的简易 CSV 行解析 */
function parseCsvLine(line) {
  const out = [];
  let cur = '';
  let inQuote = false;
  for (let i = 0; i < line.length; i++) {
    const ch = line[i];
    if (inQuote) {
      if (ch === '"') {
        if (line[i + 1] === '"') { cur += '"'; i++; } else inQuote = false;
      } else cur += ch;
    } else if (ch === '"') {
      inQuote = true;
    } else if (ch === ',') {
      out.push(cur); cur = '';
    } else {
      cur += ch;
    }
  }
  out.push(cur);
  return out.map((s) => s.trim());
}

/** 在前 80 行里找表头（必须包含指定列名） */
function findHeaderIndex(lines, required) {
  const limit = Math.min(lines.length, 80);
  for (let i = 0; i < limit; i++) {
    const cells = parseCsvLine(lines[i]);
    if (required.every((k) => cells.some((c) => c.includes(k)))) return i;
  }
  return -1;
}
```

```
const cleanNum = (s) =>
  String(s || '')
    .replace(/[¥,，\s"元]/g, '')
    .trim();

/** 收/支 列 -> income | expense | skip */
function mapDirection(s) {
  const v = String(s || '').trim();
  if (v.includes('收入') || v === '收') return 'income';
  if (v.includes('支出') || v === '支') return 'expense';
  return 'skip'; // "不计收支" / 空
}

/** "2026-06-15 12:34:56" -> "2026-06-15" */
function toDate(s) {
  const m = String(s || '').match(/(\d{4})[-/](\d{1,2})[-/](\d{1,2})/);
  if (!m) return null;
  return `${m[1]}-${String(m[2]).padStart(2, '0')}-${String(m[3]).padStart(2, '0')}`;
}

const PLATFORM_COLUMNS = {
  wechat: {
    required: ['交易时间', '金额'],
    time: ['交易时间'],
    dir: ['收/支'],
    amount: ['金额(元)', '金额'],
    item: ['商品'],
    party: ['交易对方'],
    note: ['备注'],
  },
  alipay: {
    required: ['交易时间', '金额'],
    time: ['交易时间'],
    dir: ['收/支'],
    amount: ['金额'],
    item: ['商品说明', '商品名称'],
    party: ['交易对方'],
    note: ['备注'],
  },
};

function pick(cells, header, names) {
  for (const n of names) {
    const idx = header.findIndex((h) => h === n || h.includes(n));
    if (idx >= 0 && cells[idx] !== undefined && cells[idx] !== '') return cells[idx];
  }
  return '';
}

function parseBill(buffer, platform = 'wechat') {
  const cfg = PLATFORM_COLUMNS[platform] || PLATFORM_COLUMNS.wechat;
  const text = decodeBuffer(buffer);
  const lines = splitLines(text);

  const hIdx = findHeaderIndex(lines, cfg.required);
  if (hIdx < 0) {
    const err = new Error('找不到表头行，请确认上传的是微信/支付宝导出的原始账单 CSV');
    err.status = 400; err.code = 'bad_csv';
    throw err;
  }
  const header = parseCsvLine(lines[hIdx]);

  const rows = [];
  let skipped = 0;
  for (let i = hIdx + 1; i < lines.length; i++) {
```

```
    const line = lines[i];
    if (!line || !line.trim()) continue;
    const cells = parseCsvLine(line);
    // 跳过表尾汇总行
    if (line.includes('------') || /^总|合计|汇总/.test(cells[0] || '')) { skipped++; continue; }

    const dir = mapDirection(pick(cells, header, cfg.dir));
    const date = toDate(pick(cells, header, cfg.time));
    const amount = cleanNum(pick(cells, header, cfg.amount));

    if (dir === 'skip' || !date || !amount) { skipped++; continue; }
    const cents = Math.round(Number(amount) * 100);
    if (!Number.isFinite(cents) || cents === 0) { skipped++; continue; }

    const item = pick(cells, header, cfg.item);
    const party = pick(cells, header, cfg.party);
    rows.push({
      date,
      type: dir,
      amount_cents: Math.abs(cents),
      summary: item || party || '未命名',
      counterparty: party,
      note: pick(cells, header, cfg.note),
      platform,
    });
  }

  return { rows, headerLine: hIdx + 1, skipped, encoding: detectEncodingName(buffer) };
}

function detectEncodingName(buf) {
  try {
    new TextDecoder('utf-8', { fatal: true }).decode(buf.subarray(0, 4096));
    return 'utf-8';
  } catch {
    return 'gbk';
  }
}

module.exports = { parseBill, decodeBuffer, parseCsvLine };
```

4.12 `src/lib/classifier.js`（46 行）

```
'use strict';
/**
 * 自动分类：关键词 -> 科目。
 * 对应效果图「自动分类 452 笔 / 待确认 34 笔」——
 * 命中的入 confirmed，没命中的入 pending 等人工确认。
 *
 * 上线后建议：把用户的人工确认结果回流成样本，再换成模型分类。
 */
const RULES = [
  {
    name: '原材料',
    keywords: ['原料', '奶茶', '茶包', '茶叶', '咖啡豆', '咖啡粉', '牛奶', '奶精', '果酱', '糖浆',
      '珍珠', '椰果', '杯子', '杯盖', '吸管', '打包', '包材', '食材', '面粉', '奶油', '水果', '冰块'],
  },
  {
    name: '人工',
    keywords: ['工资', '薪资', '社保', '公积金', '兼职', '小时工', '提成', '奖金', '劳务'],
  },
  {
    name: '房租',
```

```
    keywords: ['房租', '租金', '物业', '水电', '电费', '水费', '燃气'],
  },
  {
    name: '其他',
    keywords: ['维修', '装修', '快递', '运费', '广告', '推广', '团建', '办公', '话费', '宽带', '保险'],
  },
];

/** 收入侧关键词 */
const INCOME_RULES = [
  { name: '营业收入', keywords: ['营业额', '收款', '收款到账', '二维码收款', '销售'] },
  { name: '退款', keywords: ['退款', '退货'] },
];

function classify(row) {
  const text = `${row.summary || ''} ${row.counterparty || ''}`;
  const table = row.type === 'income' ? INCOME_RULES : RULES;
  for (const r of table) {
    if (r.keywords.some((k) => text.includes(k))) {
      return { category: r.name, matched: true };
    }
  }
  return { category: row.type === 'income' ? '营业收入' : '其他', matched: false };
}

module.exports = { classify, RULES, INCOME_RULES };
```

4.13 `src/lib/auth.js`（48 行）

```
'use strict';
/**
 * 极简认证：HMAC 签名 token（无状态，重启不失效）。
 * 生产环境建议换成 JWT + 刷新令牌，但演示足够。
 */
const crypto = require('node:crypto');
const { db } = require('../db');

const SECRET = process.env.AUTH_SECRET || 'dev-only-secret-please-change';

function hashPassword(pw) {
  return crypto.createHash('sha256').update(String(pw) + SECRET).digest('hex');
}

function signToken(userId) {
  const sig = crypto.createHmac('sha256', SECRET).update(String(userId)).digest('hex').slice(0, 32);
  return `${userId}.${sig}`;
}

function verifyToken(token) {
  const [id, sig] = String(token || '').split('.');
  if (!id || !sig) return null;
  const expect = crypto.createHmac('sha256', SECRET).update(String(id)).digest('hex').slice(0, 32);
  if (sig.length !== expect.length) return null;
  if (!crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expect))) return null;
  return Number(id);
}

function requireAuth(req, res, next) {
  const h = req.headers.authorization || '';
  const token = h.startsWith('Bearer ') ? h.slice(7) : null;
  const userId = verifyToken(token);
  if (!userId) return res.status(401).json({ error: 'unauthorized', message: '请先登录' });
  const user = db.prepare('SELECT id,name,phone,role,industry FROM users WHERE id=?').get(userId);
```

```
  if (!user) return res.status(401).json({ error: 'unauthorized' });
  req.user = user;
  next();
}

/** 只有老板能改系统设置/成员 */
function requireOwner(req, res, next) {
  if (req.user.role !== 'owner') {
    return res.status(403).json({ error: 'forbidden', message: '需要老板权限' });
  }
  next();
}

module.exports = { hashPassword, signToken, verifyToken, requireAuth, requireOwner };
```

4.14 `src/routes/auth.js`（34 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { hashPassword, signToken, requireAuth } = require('../lib/auth');

const router = express.Router();

/** POST /api/auth/login  { phone | name, password } */
router.post('/login', (req, res) => {
  const { phone, name, password } = req.body || {};
  if (!password) return res.status(400).json({ error: 'bad_request', message: '请输入密码' });

  const user = phone
    ? db.prepare('SELECT * FROM users WHERE phone=?').get(String(phone))
    : db.prepare('SELECT * FROM users WHERE name=?').get(String(name || ''));

  if (!user) return res.status(401).json({ error: 'unauthorized', message: '用户不存在' });
  if (user.password_hash !== hashPassword(password)) {
    return res.status(401).json({ error: 'unauthorized', message: '密码错误' });
  }

  res.json({
    token: signToken(user.id),
    user: { id: user.id, name: user.name, role: user.role, phone: user.phone, industry: user.industry
},
  });
});

/** GET /api/auth/me */
router.get('/me', requireAuth, (req, res) => res.json({ user: req.user }));

/** POST /api/auth/logout —— token 是无状态的，前端删掉即可 */
router.post('/logout', requireAuth, (req, res) => res.json({ ok: true }));

module.exports = router;
```

4.15 `src/routes/transactions.js`（169 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');
const { toCents, toYuan } = require('../lib/money');
const { monthRange, pendingCount } = require('../lib/metrics');
```

```
const router = express.Router();
router.use(requireAuth);

const rowToApi = (r) => ({
  id: r.id,
  date: r.date,
  type: r.type,
  amount: toYuan(r.amount_cents),
  category: r.category_name || null,
  categoryId: r.category_id,
  summary: r.summary,
  note: r.note,
  source: r.source,
  status: r.status,
});

/* ⚠️ 注意顺序：具体路径必须写在 /:id 前面，否则 'summary' 会被当成 id */

/** GET /api/transactions/summary?month=2026-06 —— 记账管理页顶部四张卡 */
router.get('/summary', (req, res) => {
  const month = req.query.month || new Date().toISOString().slice(0, 7);
  const [from, to] = monthRange(month);
  const base = `FROM transactions WHERE user_id=? AND status='confirmed' AND date>=? AND date<?`;
  const inc = db.prepare(`SELECT COALESCE(SUM(amount_cents),0) s ${base} AND
type='income'`).get(req.user.id, from, to);
  const exp = db.prepare(`SELECT COALESCE(SUM(amount_cents),0) s ${base} AND
type='expense'`).get(req.user.id, from, to);

  const revenue = Number(inc.s), cost = Number(exp.s);
  res.json({
    month,
    revenue: toYuan(revenue),
    cost: toYuan(cost),
    profit: toYuan(revenue - cost),
    margin: revenue ? Number(((revenue - cost) / revenue * 100).toFixed(1)) : 0,
    pendingCount: pendingCount(db, req.user.id, month),
  });
});

/** GET /api/transactions/pending —— 待确认列表 */
router.get('/pending', (req, res) => {
  const limit = Math.min(Number(req.query.size) || 50, 200);
  const rows = db
    .prepare(
      `SELECT t.*, c.name AS category_name FROM transactions t
         LEFT JOIN categories c ON c.id=t.category_id
        WHERE t.user_id=? AND t.status='pending'
        ORDER BY t.date DESC, t.id DESC LIMIT ?`
    )
    .all(req.user.id, limit);
  res.json({ items: rows.map(rowToApi) });
});

/** GET /api/transactions?month=&type=&category=&source=&page=&size= */
router.get('/', (req, res) => {
  const { month, type, category, source } = req.query;
  const page = Math.max(Number(req.query.page) || 1, 1);
  const size = Math.min(Math.max(Number(req.query.size) || 20, 1), 200);

  const where = ['t.user_id=?'];
  const args = [req.user.id];
  // 默认只返回已确认的记录；?status=all 看全部，?status=pending 看待确认
  const status = req.query.status || 'confirmed';
  if (status !== 'all') { where.push('t.status=?'); args.push(status); }
```

```
  if (month) { const [f, t] = monthRange(month); where.push('t.date>=?', 't.date<?'); args.push(f,
t); }
  if (type) { where.push('t.type=?'); args.push(type); }
  if (source) { where.push('t.source=?'); args.push(source); }
  if (category) { where.push('c.name=?'); args.push(category); }

  const w = where.join(' AND ');
  const total = Number(
    db.prepare(`SELECT COUNT(*) c FROM transactions t LEFT JOIN categories c ON c.id=t.category_id
WHERE ${w}`).get(...args).c
  );
  const rows = db
    .prepare(
      `SELECT t.*, c.name AS category_name FROM transactions t
         LEFT JOIN categories c ON c.id=t.category_id
        WHERE ${w} ORDER BY t.date DESC, t.id DESC LIMIT ? OFFSET ?`
    )
    .all(...args, size, (page - 1) * size);

  res.json({ items: rows.map(rowToApi), total, page, size });
});

/** POST /api/transactions —— 手工录入 */
router.post('/', (req, res) => {
  const { date, type, amount, category, summary, note, source } = req.body || {};
  if (!date || !type || amount === undefined) {
    return res.status(400).json({ error: 'bad_request', message: 'date / type / amount 必填' });
  }
  if (!['income', 'expense'].includes(type)) {
    return res.status(400).json({ error: 'bad_request', message: 'type 只能是 income 或 expense' });
  }

  let categoryId = null;
  if (category) {
    const c = db.prepare('SELECT id FROM categories WHERE user_id=? AND name=?').get(req.user.id,
category);
    categoryId = c ? c.id : null;
    if (!categoryId) {
      const info = db.prepare('INSERT INTO categories(user_id,name,kind)
VALUES(?,?,?)').run(req.user.id, category, type);
      categoryId = Number(info.lastInsertRowid);
    }
  }

  const info = db
    .prepare(
      `INSERT INTO
transactions(user_id,date,type,amount_cents,category_id,summary,note,source,status)
       VALUES(?,?,?,?,?,?,?,?,'confirmed')`
    )
    .run(req.user.id, date, type, toCents(amount), categoryId, summary || null, note || null, source
|| 'manual');

  const row = db.prepare('SELECT * FROM transactions WHERE id=?').get(Number(info.lastInsertRowid));
  res.status(201).json(rowToApi(row));
});

/** POST /api/transactions/:id/confirm —— 人工确认归类 */
router.post('/:id/confirm', (req, res) => {
  const id = Number(req.params.id);
  const { category } = req.body || {};
  const row = db.prepare('SELECT * FROM transactions WHERE id=? AND user_id=?').get(id, req.user.id);
  if (!row) return res.status(404).json({ error: 'not_found' });

  let categoryId = row.category_id;
```

```
  if (category) {
    let c = db.prepare('SELECT id FROM categories WHERE user_id=? AND name=?').get(req.user.id,
category);
    if (!c) {
      const info = db.prepare('INSERT INTO categories(user_id,name,kind)
VALUES(?,?,?)').run(req.user.id, category, row.type);
      categoryId = Number(info.lastInsertRowid);
    } else categoryId = c.id;
  }
  db.prepare(`UPDATE transactions SET status='confirmed', category_id=? WHERE id=?`).run(categoryId,
id);
  const updated = db.prepare('SELECT * FROM transactions WHERE id=?').get(id);
  res.json(rowToApi(updated));
});

/** PUT /api/transactions/:id */
router.put('/:id', (req, res) => {
  const id = Number(req.params.id);
  const row = db.prepare('SELECT * FROM transactions WHERE id=? AND user_id=?').get(id, req.user.id);
  if (!row) return res.status(404).json({ error: 'not_found' });

  const { date, type, amount, summary, note } = req.body || {};
  db.prepare(
    `UPDATE transactions SET date=?, type=?, amount_cents=?, summary=?, note=? WHERE id=?`
  ).run(
    date ?? row.date,
    type ?? row.type,
    amount !== undefined ? toCents(amount) : row.amount_cents,
    summary ?? row.summary,
    note ?? row.note,
    id
  );
  res.json(rowToApi(db.prepare('SELECT * FROM transactions WHERE id=?').get(id)));
});

/** DELETE /api/transactions/:id */
router.delete('/:id', (req, res) => {
  const id = Number(req.params.id);
  const info = db.prepare('DELETE FROM transactions WHERE id=? AND user_id=?').run(id, req.user.id);
  if (!info.changes) return res.status(404).json({ error: 'not_found' });
  res.json({ ok: true, deleted: id });
});

module.exports = router;
```

4.16 `src/routes/import.js`（105 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');
const { parseBill } = require('../lib/csv');
const { classify } = require('../lib/classifier');

const router = express.Router();
router.use(requireAuth);

function getOrCreateCategory(userId, name, kind) {
  const hit = db.prepare('SELECT id FROM categories WHERE user_id=? AND name=?').get(userId, name);
  if (hit) return hit.id;
  const info = db.prepare('INSERT INTO categories(user_id,name,kind) VALUES(?,?,?)').run(userId,
name, kind);
  return Number(info.lastInsertRowid);
```

```
}

/**
 * POST /api/import/bills?platform=wechat|alipay
 * Body: 账单 CSV 的原始字节（Content-Type: application/octet-stream 或 text/plain）
 *
 * 返回：{ jobId, total, categorized, pending, skipped, encoding }
 * 正好对应「数据采集」页的「共 486 笔 / 自动分类 452 笔 / 待确认 34 笔」
 */
router.post('/bills', (req, res) => {
  const platform = String(req.query.platform || req.headers['x-platform'] || 'wechat').toLowerCase();
  if (!['wechat', 'alipay'].includes(platform)) {
    return res.status(400).json({ error: 'bad_request', message: 'platform 只能是 wechat 或 alipay'
});
  }

  const buf = Buffer.isBuffer(req.body) ? req.body : Buffer.from(req.body || '');
  if (!buf.length) {
    return res.status(400).json({ error: 'bad_request', message: '请求体为空，请把账单 CSV 内容作为原始字节
上传' });
  }

  let parsed;
  try {
    parsed = parseBill(buf, platform);
  } catch (e) {
    return res.status(e.status || 400).json({ error: e.code || 'bad_csv', message: e.message });
  }

  let categorized = 0;
  let pending = 0;
  const insert = db.prepare(
    `INSERT INTO transactions(user_id,date,type,amount_cents,category_id,summary,note,source,status)
     VALUES(?,?,?,?,?,?,?,?,?)`
  );

  db.exec('BEGIN');
  try {
    for (const r of parsed.rows) {
      const c = classify(r);
      const catId = getOrCreateCategory(req.user.id, c.category, r.type);
      const status = c.matched ? 'confirmed' : 'pending';
      insert.run(
        req.user.id, r.date, r.type, r.amount_cents, catId,
        r.summary, r.note || null, platform, status
      );
      if (c.matched) categorized++; else pending++;
    }
    db.exec('COMMIT');
  } catch (e) {
    db.exec('ROLLBACK');
    throw e;
  }

  const info = db
    .prepare(
      `INSERT INTO import_jobs(user_id,platform,filename,total,categorized,pending,status,message)
       VALUES(?,?,?,?,?,?,'done',?)`
    )
    .run(req.user.id, platform, req.headers['x-filename'] || null,
      parsed.rows.length, categorized, pending,
      `编码 ${parsed.encoding}，表头在第 ${parsed.headerLine} 行，跳过 ${parsed.skipped} 行`);

  res.status(201).json({
    jobId: Number(info.lastInsertRowid),
```

```
    total: parsed.rows.length,
    categorized,
    pending,
    skipped: parsed.skipped,
    encoding: parsed.encoding,
    headerLine: parsed.headerLine,
  });
});

/** GET /api/import —— 导入历史 */
router.get('/', (req, res) => {
  const rows = db
    .prepare('SELECT * FROM import_jobs WHERE user_id=? ORDER BY id DESC LIMIT 20')
    .all(req.user.id);
  res.json({ items: rows });
});

/** GET /api/import/:jobId —— 导入结果 */
router.get('/:jobId', (req, res) => {
  const job = db
    .prepare('SELECT * FROM import_jobs WHERE id=? AND user_id=?')
    .get(Number(req.params.jobId), req.user.id);
  if (!job) return res.status(404).json({ error: 'not_found' });
  res.json(job);
});

module.exports = router;
```

4.17 `src/routes/categories.js`（34 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');

const router = express.Router();
router.use(requireAuth);

/** GET /api/categories?kind=expense */
router.get('/', (req, res) => {
  const { kind } = req.query;
  const rows = kind
    ? db.prepare('SELECT * FROM categories WHERE user_id=? AND kind=? ORDER BY id').all(req.user.id,
kind)
    : db.prepare('SELECT * FROM categories WHERE user_id=? ORDER BY kind, id').all(req.user.id);
  res.json({ items: rows });
});

/** POST /api/categories  { name, kind } */
router.post('/', (req, res) => {
  const { name, kind = 'expense', color } = req.body || {};
  if (!name) return res.status(400).json({ error: 'bad_request', message: 'name 必填' });
  if (!['income', 'expense'].includes(kind)) {
    return res.status(400).json({ error: 'bad_request', message: 'kind 只能是 income 或 expense' });
  }
  const exist = db.prepare('SELECT * FROM categories WHERE user_id=? AND name=?').get(req.user.id,
name);
  if (exist) return res.status(409).json({ error: 'conflict', message: '分类已存在', item: exist });

  const info = db
    .prepare('INSERT INTO categories(user_id,name,kind,color) VALUES(?,?,?,?)')
    .run(req.user.id, name, kind, color || null);
  res.status(201).json(db.prepare('SELECT * FROM categories WHERE
```

```
id=?').get(Number(info.lastInsertRowid)));
});

module.exports = router;
```

4.18 `src/routes/products.js`（58 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');
const { toCents, toYuan, pct } = require('../lib/money');

const router = express.Router();
router.use(requireAuth);

const api = (p) => ({
  id: p.id,
  name: p.name,
  price: toYuan(p.price_cents),
  cost: toYuan(p.cost_cents),
  margin: pct(p.price_cents - p.cost_cents, p.price_cents),
});

/** GET /api/products */
router.get('/', (req, res) => {
  const rows = db.prepare('SELECT * FROM products WHERE user_id=? ORDER BY id').all(req.user.id);
  res.json({ items: rows.map(api) });
});

/** POST /api/products  { name, price, cost } */
router.post('/', (req, res) => {
  const { name, price, cost } = req.body || {};
  if (!name || price === undefined || cost === undefined) {
    return res.status(400).json({ error: 'bad_request', message: 'name / price / cost 必填' });
  }
  const info = db
    .prepare('INSERT INTO products(user_id,name,price_cents,cost_cents) VALUES(?,?,?,?)')
    .run(req.user.id, name, toCents(price), toCents(cost));
  res.status(201).json(api(db.prepare('SELECT * FROM products WHERE
id=?').get(Number(info.lastInsertRowid))));
});

/** PUT /api/products/:id */
router.put('/:id', (req, res) => {
  const id = Number(req.params.id);
  const row = db.prepare('SELECT * FROM products WHERE id=? AND user_id=?').get(id, req.user.id);
  if (!row) return res.status(404).json({ error: 'not_found' });
  const { name, price, cost } = req.body || {};
  db.prepare('UPDATE products SET name=?, price_cents=?, cost_cents=? WHERE id=?').run(
    name ?? row.name,
    price !== undefined ? toCents(price) : row.price_cents,
    cost !== undefined ? toCents(cost) : row.cost_cents,
    id
  );
  res.json(api(db.prepare('SELECT * FROM products WHERE id=?').get(id)));
});

/** DELETE /api/products/:id */
router.delete('/:id', (req, res) => {
  const info = db.prepare('DELETE FROM products WHERE id=? AND user_id=?').run(Number(req.params.id),
req.user.id);
  if (!info.changes) return res.status(404).json({ error: 'not_found' });
```

```
  res.json({ ok: true });
});

module.exports = router;
```

4.19 `src/routes/analytics.js`（48 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');
const metrics = require('../lib/metrics');

const router = express.Router();
router.use(requireAuth);

const curMonth = () => new Date().toISOString().slice(0, 7);

/** GET /api/analytics/overview?month=2026-06 —— 四张卡 + 环比 + 同比 */
router.get('/overview', (req, res) => {
  res.json(metrics.overview(db, req.user.id, req.query.month || curMonth()));
});

/** GET /api/analytics/trend?from=2026-01&to=2026-06 —— 营收趋势折线 */
router.get('/trend', (req, res) => {
  const to = req.query.to || curMonth();
  const from = req.query.from || metrics.shiftMonth(to, -5);
  res.json({ from, to, items: metrics.trend(db, req.user.id, from, to) });
});

/** GET /api/analytics/cost-structure?month=2026-06 —— 成本结构饼图 */
router.get('/cost-structure', (req, res) => {
  res.json(metrics.costStructure(db, req.user.id, req.query.month || curMonth()));
});

/** GET /api/analytics/product-margin —— 商品毛利条 */
router.get('/product-margin', (req, res) => {
  res.json({ items: metrics.productMargin(db, req.user.id) });
});

/** GET /api/analytics/compare?month=2026-06 —— 同环比表格 */
router.get('/compare', (req, res) => {
  const o = metrics.overview(db, req.user.id, req.query.month || curMonth());
  const rows = [
    { metric: '营业额', current: o.revenue, prev: o.prev.revenue },
    { metric: '成本', current: o.cost, prev: o.prev.cost },
    { metric: '利润', current: o.profit, prev: o.prev.profit },
  ].map((r) => ({
    ...r,
    mom: r.prev ? Number(((r.current - r.prev) / r.prev * 100).toFixed(1)) : null,
  }));
  res.json({ month: o.month, prevMonth: o.prev.month, items: rows, yoy: o.yoy });
});

module.exports = router;
```

4.20 `src/routes/ai.js`（167 行）

```
'use strict';
/**
 * 智能问答 —— PRD 的核心创新点：**系统算数、模型解释**。
```

```
 *
 * 流程：
 *   1. 后端先把数字算好（metrics.overview）
 *   2. 把数字作为「计算依据」放进提示词上下文
 *   3. 调用大模型，只让它做归因和语言组织
 *   4. 返回 answer + basis，前端右侧直接展示 basis
 *
 * 没有配置 LLM_API_KEY 时自动降级为「内置规则引擎」，
 * 演示时不依赖网络，也不会因为是断网就讲不了。
 */
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');
const metrics = require('../lib/metrics');

const router = express.Router();
router.use(requireAuth);

const curMonth = () => new Date().toISOString().slice(0, 7);
const fd = (n) => (n === null || n === undefined ? '—' : `${n > 0 ? '+' : ''}${n}%`);

/** 把指标整理成「计算依据」数组（前端右侧那块） */
function buildBasis(o) {
  return [
    { label: '营业额', month: o.month, current: o.revenue, prev: o.prev.revenue, mom: o.mom.revenue },
    { label: '成本', month: o.month, current: o.cost, prev: o.prev.cost, mom: o.mom.cost },
    { label: '利润', month: o.month, current: o.profit, prev: o.prev.profit, mom: o.mom.profit },
    { label: '毛利率', month: o.month, current: o.margin, prev: o.prev.margin, mom: o.mom.margin },
  ];
}

/** 生成提示词上下文（数字全部来自系统计算） */
function buildPrompt(o) {
  return [
    `本月(${o.month})：营业额 ${o.revenue} 元，成本 ${o.cost} 元，利润 ${o.profit} 元，毛利率
${o.margin}%。`,
    `上月(${o.prev.month})：营业额 ${o.prev.revenue} 元，成本 ${o.prev.cost} 元，利润 ${o.prev.profit}
元，毛利率 ${o.prev.margin}%。`,
    `环比：营业额 ${fd(o.mom.revenue)}，成本 ${fd(o.mom.cost)}，利润 ${fd(o.mom.profit)}，毛利率
${o.mom.margin} 个百分点。`,
    `同比：营业额 ${fd(o.yoy.revenue)}，利润 ${fd(o.yoy.profit)}。`,
    `待确认账单 ${o.pendingCount} 笔。`,
  ].join('\n');
}

const SYSTEM_PROMPT = `你是小微企业经营管理助手。
规则：
1. 所有数字都由系统计算好后提供给你，你只能引用，绝不能自己计算或编造数字。
2. 用中文回答，先说原因（3 条以内），再给 1 条可执行建议。
3. 语言简洁，面向没有财务基础的店主。
4. 不要输出 Markdown 表格。`;

/** 无 API Key 时的兜底：内置规则引擎 */
function ruleBasedAnswer(o, costDelta, question) {
  const reasons = [];

  // 1) 收入侧
  if (o.mom.revenue !== null && o.mom.revenue < 0) {
    reasons.push(`营业额下降 ${Math.abs(o.mom.revenue)}%`);
  } else if (o.mom.revenue > 0) {
    reasons.push(`营业额上升 ${o.mom.revenue}%`);
  }

  // 2) 成本侧：找涨幅最大的成本项（分类级，不是总额）
  const rising = costDelta
```

```
    .filter((c) => c.mom !== null && c.mom > 0 && c.prev > 0)
    .sort((a, b) => b.mom - a.mom)[0];
  if (rising) {
    reasons.push(`${rising.name}成本上涨 ${rising.mom}%`);
  } else if (o.mom.cost !== null && o.mom.cost > 0) {
    reasons.push(`总成本上涨 ${o.mom.cost}%`);
  }

  // 3) 利润质量
  if (o.mom.margin < 0) {
    reasons.push(`毛利率下降 ${Math.abs(o.mom.margin)} 个点`);
  } else if (o.mom.margin > 0) {
    reasons.push(`毛利率提升 ${o.mom.margin} 个点`);
  }

  if (!reasons.length) reasons.push('本月各项指标与上月基本持平');

  let suggestion = '建议保持当前成本结构，继续观察 2 周趋势。';
  if (o.mom.revenue < 0) {
    suggestion = '营业额下滑，建议排查客流变化，并针对高毛利商品做组合推荐。';
  }
  if (rising && o.mom.revenue <= 0) {
    suggestion = `${rising.name}成本上涨而收入未同步增长，建议先与该类供应商议价，或调整套餐结构提高客单价。`;
  }

  return {
    answer: `主要有 ${reasons.length} 个原因：\n` + reasons.map((r) => `• ${r}`).join('\n'),
    suggestion,
  };
}

/** POST /api/ai/ask  { question, month } */
router.post('/ask', async (req, res, next) => {
  const { question, month } = req.body || {};
  if (!question || !String(question).trim()) {
    return res.status(400).json({ error: 'bad_request', message: 'question 不能为空' });
  }

  const o = metrics.overview(db, req.user.id, month || curMonth());
  const basis = buildBasis(o);
  const context = buildPrompt(o);

  // 成本结构也可以作为补充上下文（按需拼进去）
  const cs = metrics.costStructure(db, req.user.id, o.month);
  const costDelta = metrics.categorySeries(db, req.user.id, o.month, o.prev.month);
  const csText = cs.items.map((i) => `${i.name} ${i.ratio}%`).join('，');

  const apiKey = process.env.LLM_API_KEY;
  let engine = 'rule-based';
  let answer, suggestion;

  if (apiKey) {
    try {
      const base = process.env.LLM_BASE_URL || 'https://api.deepseek.com/v1';
      const model = process.env.LLM_MODEL || 'deepseek-chat';
      const resp = await fetch(`${base.replace(/\/$/, '')}/chat/completions`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${apiKey}` },
        body: JSON.stringify({
          model,
          temperature: 0.3,
          messages: [
            { role: 'system', content: SYSTEM_PROMPT },
            {
              role: 'user',
```

```
              content: `【系统已计算的真实数据】\n${context}\n\n【成本结构】${csText}\n\n【用户问题】
${question}\n\n请只基于上面的数据回答，不要编造数字。`,
            },
          ],
        }),
        signal: AbortSignal.timeout(Number(process.env.LLM_TIMEOUT_MS || 20000)),
      });
      if (!resp.ok) throw new Error(`LLM HTTP ${resp.status}`);
      const data = await resp.json();
      const text = data?.choices?.[0]?.message?.content?.trim();
      if (!text) throw new Error('LLM 返回为空');
      answer = text;
      suggestion = null;
      engine = model;
    } catch (e) {
      console.warn('[ai] 大模型调用失败，降级为规则引擎:', e.message);
      const fb = ruleBasedAnswer(o, costDelta, question);
      answer = fb.answer; suggestion = fb.suggestion;
    }
  } else {
    const fb = ruleBasedAnswer(o, costDelta, question);
    answer = fb.answer; suggestion = fb.suggestion;
  }

  res.json({
    month: o.month,
    question,
    answer,
    suggestion,
    basis,
    costStructure: cs,
    costDelta,
    engine,
    actions: ['看细节', '对比上月', '出报告'],
  });
});

module.exports = router;
```

4.21 `src/routes/alerts.js`（52 行）

```
'use strict';
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');

const router = express.Router();
router.use(requireAuth);

/** GET /api/alerts —— 预警列表（?status=open） */
router.get('/', (req, res) => {
  const status = req.query.status || 'open';
  const rows = db
    .prepare('SELECT * FROM alerts WHERE user_id=? AND status=? ORDER BY CASE level WHEN \'high\'
THEN 0 WHEN \'medium\' THEN 1 ELSE 2 END, id DESC')
    .all(req.user.id, status);
  const open = Number(db.prepare(`SELECT COUNT(*) c FROM alerts WHERE user_id=? AND
status='open'`).get(req.user.id).c);
  res.json({ items: rows, openCount: open });
});

/** GET /api/alerts/settings —— ⚠️ 必须写在 /:id 之前 */
router.get('/settings', (req, res) => {
```

```
  const s = db.prepare('SELECT * FROM alert_settings WHERE user_id=?').get(req.user.id);
  res.json(s || { user_id: req.user.id, cost_ratio: 40, revenue_volatility: 5, margin_floor: 30 });
});

/** PUT /api/alerts/settings  { cost_ratio, revenue_volatility, margin_floor } */
router.put('/settings', (req, res) => {
  const { cost_ratio, revenue_volatility, margin_floor } = req.body || {};
  const cur = db.prepare('SELECT * FROM alert_settings WHERE user_id=?').get(req.user.id);
  if (!cur) {
    db.prepare('INSERT INTO alert_settings(user_id,cost_ratio,revenue_volatility,margin_floor)
VALUES(?,?,?,?)')
      .run(req.user.id, cost_ratio ?? 40, revenue_volatility ?? 5, margin_floor ?? 30);
  } else {
    db.prepare('UPDATE alert_settings SET cost_ratio=?, revenue_volatility=?, margin_floor=? WHERE
user_id=?')
      .run(cost_ratio ?? cur.cost_ratio, revenue_volatility ?? cur.revenue_volatility, margin_floor
?? cur.margin_floor, req.user.id);
  }
  res.json(db.prepare('SELECT * FROM alert_settings WHERE user_id=?').get(req.user.id));
});

/** POST /api/alerts/:id/ignore */
router.post('/:id/ignore', (req, res) => {
  const info = db.prepare(`UPDATE alerts SET status='ignored' WHERE id=? AND user_id=?
`).run(Number(req.params.id), req.user.id);
  if (!info.changes) return res.status(404).json({ error: 'not_found' });
  res.json({ ok: true });
});

/** GET /api/alerts/unread-count —— 侧边栏小红点 */
router.get('/unread-count', (req, res) => {
  const c = Number(db.prepare(`SELECT COUNT(*) c FROM alerts WHERE user_id=? AND
status='open'`).get(req.user.id).c);
  res.json({ count: c });
});

module.exports = router;
```

4.22 `src/routes/ocr.js`（85 行）

```
'use strict';
/**
 * 票据识别（OCR）—— 当前是**占位实现**。
 *
 * 为什么先做占位：
 *   前端需要「上传图片 -> 拿到识别结果 -> 确认入账」这条链路能跑通。
 *   真 OCR（华为云文字识别 / 腾讯云 OCR）需要开通服务 + 鉴权 + 调额度，
 *   先把链路打通，再换实现，前端完全不用改。
 *
 * 换成真 OCR 时只改 recognize() 这一个函数。
 */
const express = require('express');
const { db } = require('../db');
const { requireAuth } = require('../lib/auth');
const { toYuan } = require('../lib/money');

const router = express.Router();
router.use(requireAuth);

/** 占位识别：返回和效果图一致的结果结构 */
async function recognize(buffer, filename) {
  return {
    merchant: '悦茶饮品店',
```

```
    date: '2026-06-15',
    amount: 128.5,
    category: '原材料',
    note: '珍珠奶茶 ×2',
    confidence: { merchant: 98, date: 99, amount: 97, category: 96, note: 94 },
    _placeholder: true,
  };
}

/** POST /api/ocr/receipt —— Body 为图片原始字节 */
router.post('/receipt', async (req, res, next) => {
  try {
    const buf = Buffer.isBuffer(req.body) ? req.body : Buffer.alloc(0);
    if (!buf.length) return res.status(400).json({ error: 'bad_request', message: '请上传图片（Content-
Type: application/octet-stream 或 image/*）' });

    const r = await recognize(buf, req.headers['x-filename'] || null);
    const info = db
      .prepare(
        `INSERT INTO
receipts(user_id,image_path,merchant,date,amount_cents,category,note,confidence_json,status)
         VALUES(?,?,?,?,?,?,?,?,'pending')`
      )
      .run(req.user.id, req.headers['x-filename'] || null, r.merchant, r.date,
        Math.round(r.amount * 100), r.category, r.note, JSON.stringify(r.confidence));

    res.status(201).json({ id: Number(info.lastInsertRowid), ...r, amount: toYuan(Math.round(r.amount
* 100)) });
  } catch (e) { next(e); }
});

/** POST /api/ocr/receipt/:id/confirm —— 确认入账 */
router.post('/receipt/:id/confirm', (req, res) => {
  const id = Number(req.params.id);
  const r = db.prepare('SELECT * FROM receipts WHERE id=? AND user_id=?').get(id, req.user.id);
  if (!r) return res.status(404).json({ error: 'not_found' });
  if (r.status === 'confirmed') return res.status(409).json({ error: 'conflict', message: '该票据已入
账' });

  let cat = db.prepare('SELECT id FROM categories WHERE user_id=? AND name=?').get(req.user.id,
r.category);
  let catId;
  if (!cat) {
    const info = db.prepare('INSERT INTO categories(user_id,name,kind)
VALUES(?,?,?)').run(req.user.id, r.category, 'expense');
    catId = Number(info.lastInsertRowid);
  } else catId = cat.id;

  const body = req.body || {};
  const txInfo = db
    .prepare(
      `INSERT INTO
transactions(user_id,date,type,amount_cents,category_id,summary,note,source,status)
       VALUES(?,?,'expense',?,?,?,?,'ocr','confirmed')`
    )
    .run(req.user.id, body.date || r.date, body.amount !== undefined ? Math.round(Number(body.amount)
* 100) : r.amount_cents,
      catId, body.summary || r.merchant, body.note || r.note);

  db.prepare(`UPDATE receipts SET status='confirmed' WHERE id=?`).run(id);
  res.status(201).json({ ok: true, receiptId: id, transactionId: Number(txInfo.lastInsertRowid) });
});

/** GET /api/ocr/receipts —— 票据历史 */
router.get('/receipts', (req, res) => {
```

```
  const rows = db.prepare('SELECT * FROM receipts WHERE user_id=? ORDER BY id DESC LIMIT
50').all(req.user.id);
  res.json({ items: rows.map((r) => ({ ...r, amount: toYuan(r.amount_cents) })) });
});

module.exports = router;
```

4.23 `src/routes/system.js`（95 行）

```
'use strict';
const express = require('express');
const fs = require('node:fs');
const path = require('node:path');
const { db, DB_PATH } = require('../db');
const { requireAuth, requireOwner, hashPassword } = require('../lib/auth');
const { toYuan } = require('../lib/money');
const { monthRange } = require('../lib/metrics');

const router = express.Router();
router.use(requireAuth);

/** GET /api/system/members —— 账号与权限 */
router.get('/members', (req, res) => {
  const rows = db.prepare('SELECT id,name,phone,role,industry,created_at FROM users ORDER BY
id').all();
  res.json({ items: rows });
});

/** POST /api/system/members/invite —— 仅老板 */
router.post('/members/invite', requireOwner, (req, res) => {
  const { name, phone, password = '123456', role = 'staff' } = req.body || {};
  if (!name || !phone) return res.status(400).json({ error: 'bad_request', message: 'name / phone 必
填' });
  if (!['owner', 'staff'].includes(role)) return res.status(400).json({ error: 'bad_request',
message: 'role 非法' });
  const exist = db.prepare('SELECT id FROM users WHERE phone=?').get(String(phone));
  if (exist) return res.status(409).json({ error: 'conflict', message: '该手机号已存在' });
  const info = db.prepare('INSERT INTO users(name,phone,industry,role,password_hash)
VALUES(?,?,?,?,?)')
    .run(name, String(phone), '餐饮', role, hashPassword(password));
  res.status(201).json(db.prepare('SELECT id,name,phone,role FROM users WHERE
id=?').get(Number(info.lastInsertRowid)));
});

/** PUT /api/system/members/:id —— 改角色 */
router.put('/members/:id', requireOwner, (req, res) => {
  const { role } = req.body || {};
  if (!['owner', 'staff'].includes(role)) return res.status(400).json({ error: 'bad_request',
message: 'role 非法' });
  const info = db.prepare('UPDATE users SET role=? WHERE id=?').run(role, Number(req.params.id));
  if (!info.changes) return res.status(404).json({ error: 'not_found' });
  res.json(db.prepare('SELECT id,name,phone,role FROM users WHERE id=?').get(Number(req.params.id)));
});

/** POST /api/system/backup —— 导出全库 JSON 快照到 data/backups/ */
router.post('/backup', requireOwner, (req, res) => {
  const tables = ['users', 'categories', 'transactions', 'products', 'alerts', 'alert_settings',
'receipts'];
  const snapshot = { at: new Date().toISOString(), db: DB_PATH, data: {} };
  for (const t of tables) snapshot.data[t] = db.prepare(`SELECT * FROM ${t}`).all();

  const dir = path.join(path.dirname(DB_PATH), 'backups');
  fs.mkdirSync(dir, { recursive: true });
```

```
  const file = path.join(dir, `backup-${Date.now()}.json`);
  fs.writeFileSync(file, JSON.stringify(snapshot, null, 2));

  res.json({
    ok: true,
    at: snapshot.at,
    file: path.basename(file),
    size: fs.statSync(file).size,
    tables: Object.fromEntries(Object.entries(snapshot.data).map(([k, v]) => [k, v.length])),
  });
});

/** GET /api/system/export?format=csv|xlsx|pdf&month=2026-06 */
router.get('/export', (req, res) => {
  const format = String(req.query.format || 'csv').toLowerCase();
  const month = req.query.month || new Date().toISOString().slice(0, 7);
  const [from, to] = monthRange(month);

  const rows = db
    .prepare(
      `SELECT t.date, t.type, t.amount_cents, COALESCE(c.name,'未分类') cat, t.summary, t.source,
t.status
         FROM transactions t LEFT JOIN categories c ON c.id=t.category_id
        WHERE t.user_id=? AND t.date>=? AND t.date<?
        ORDER BY t.date, t.id`
    )
    .all(req.user.id, from, to);

  const head = ['日期', '类型', '金额(元)', '分类', '摘要', '来源', '状态'];
  const lines = [head.join(',')].concat(
    rows.map((r) => [r.date, r.type === 'income' ? '收入' : '支出', toYuan(r.amount_cents), r.cat,
      `"${(r.summary || '').replace(/"/g, '""')}"`, r.source, r.status].join(','))
  );
  const csv = '\uFEFF' + lines.join('\n'); // BOM 让 Excel 正确识别中文

  if (format === 'xlsx' || format === 'pdf') {
    // 扩展项：接 exceljs / pdfkit 后可输出真格式，当前降级为 CSV
    res.setHeader('X-Warning', `${format} 尚未实现，已降级为 CSV`);
    res.setHeader('Content-Type', 'text/csv; charset=utf-8');
    res.setHeader('Content-Disposition', `attachment; filename="sme-i-${month}.csv"`);
    return res.send(csv);
  }

  res.setHeader('Content-Type', 'text/csv; charset=utf-8');
  res.setHeader('Content-Disposition', `attachment; filename="sme-i-${month}.csv"`);
  res.send(csv);
});

module.exports = router;
```